

Scheduler Affinity

Document #: P3941R4 [[Latest](#)] [[Status](#)]
Date: 2026-03-26
Project: Programming Language C++
Audience: Concurrency Working Group (SG1)
Library Evolution Working Group (LEWG)
Library Working Group (LWG)
Reply-to: Dietmar Kühl (Bloomberg)
<dkuhl@bloomberg.net>

Contents

- [1 Change History](#)
 - [1.1 R4](#)
 - [1.2 R3](#)
 - [1.3 R2](#)
 - [1.4 R1](#)
 - [1.5 R0](#)
- [2 Overview of Changes](#)
- [3 Discussion of Changes](#)
 - [3.1 affine_on Shape](#)
 - [3.2 Scheduler An Operation Was Started On](#)
 - [3.3 Infallible Schedulers](#)
 - [3.3.1 Require Infallible Schedulers For affine_on](#)
 - [3.3.2 Allow Fallible Schedulers For affine_on](#)
 - [3.3.3 Considerations On Infallible Schedulers](#)
 - [3.4 affine_on Customization](#)
 - [3.5 Removing change_coroutine_scheduler](#)
 - [3.6 affine_on Default Implementation](#)
 - [3.7 Name Change](#)
 - [3.8 Use get_start_scheduler Properly](#)
- [4 Wording Changes](#)

One important design of `std::execution::task` is that a coroutine resumes after a `co_await` on the same scheduler as the one it was executing on prior to the `co_await`. To achieve this, `task` transforms the awaited object `obj` using `affine_on(obj, sched)` where `sched` is the corresponding scheduler. There were multiple concerns raised against the specification of `affine_on` and discussed as part of [P3796R1](#). This proposal is intended to specifically address the concerns raised relating to `task`'s scheduler affinity and in particular `affine_on`. The gist of this proposal is impose constraints on `affine_on` to guarantee it can meet its objective at run-time.

§ 1 Change History

§ 1.1 R4

- remove the alternative wording using `get_scheduler`
- remove fallback `get_scheduler` for `get_start_scheduler`
- use `get_start_scheduler` instead of `get_scheduler` to refer to the scheduler an operation got started on (either for queries or for defining environments)
- use `start_scheduler_type` instead of `scheduler_type` in `task`
- give implementations permission to define `sndr.affine_on()` and `sndr.as_awaitable(p)`

§ 1.2 R3

- rebase changes on the customization changes [P3826r3](#)
- use `transform_sender` in `as_awaitable` to locate possible customization of nested senders in `[exec.as.awaitable]`

§ 1.3 R2

- added requirement on `get_scheduler/get_start_scheduler`
- fixed typo in the wording for `affine_on::transform_sender`
- fixed various typos in the text

§ 1.4 R1

- added wording

§ 1.5 R0

- initial revision

§ 2 Overview of Changes

There are a few NB comments raised about the way `affine_on` works:

- [US 232-366](#): specify customization of `affine_on` when the scheduler doesn't change.
- [US 233-365](#): clarify `affine_on` vs. `continues_on`.
- [US 234-364](#): remove scheduler parameter from `affine_on`.
- [US 235-363](#): `affine_on` should not forward the stop token to the scheduling operation.
- [US 236-362](#): specify default implementation of `affine_on`.

The discussion on `affine_on` revealed some aspects which were not quite clear previously and taking these into account points towards a better design than was previously specified:

1. To implement scheduler affinity the algorithm needs to know the scheduler on which it was started itself. The correct scheduler may actually be hard to determine while building the work graph. However, this scheduler can be communicated using `get_scheduler(get_env(rcvr))` when an algorithm is started. This requirement is more general than just `affine_on` and is introduced by [P3718R0](#): with this guarantee in place, `affine_on` only needs one parameter, i.e., the sender for the work to be executed. [P3718R0](#) is, however, discontinued (for unrelated reasons) and adding the guarantee to get the current scheduler from a receiver query is proposed here.
2. The scheduler *sched* on which the work needs to resume has to guarantee that it is possible to resume on the correct execution agent. The implication is that scheduling work needs to be infallible, i.e., the completion signatures of `scheduler(sched)` cannot contain a `set_error_t(E)` completion signature. This requirement should be checked statically.
3. The work needs to be resumed on the correct scheduler even when the work is stopped, i.e., the scheduling operation shall be connected to a receiver whose environment's `get_stop_token` query yields an `unstoppable_token`. In addition, the scheduling operation shall not have a `set_stopped_t()` completion signature if the environment's `get_stop_token` query yields an `unstoppable_token`. This requirement should also be checked statically.
4. When a sender knows that it will complete on the scheduler it was started on, it should be possible to customize the `affine_on` algorithm to avoid rescheduling. This customization can be achieved by connecting to the result of an `affine_on` member function called on the child sender, if such a member function is present, when connecting an `affine_on` sender.

None of these changes really contradict any earlier design: the shape and behavior of the `affine_on` algorithm wasn't fully fleshed out. Tightening the behavior of scheduler affinity and the `affine_on` algorithm has some implications on some other components:

1. If `affine_on` requires an infallible scheduler at least `inline_scheduler`, `task_scheduler`, and `run_loop::scheduler` should be infallible (i.e., they always

complete successfully with `set_value()`). `parallel_scheduler` can probably not be made infallible.

2. The scheduling semantics when changing a task's scheduler using `co_await change_coroutine_scheduler(sch)` become somewhat unclear and this functionality should be removed. Similar semantics are better modeled using `co_await on(sch, nested-task)`.
3. The name `affine_on` isn't particularly good and wasn't designed. It may be worth renaming the algorithms to something different.

§ 3 Discussion of Changes

§ 3.1 `affine_on` Shape

The original proposal for task used `continues_on` to schedule the work back on the original scheduler. This algorithm takes the work to be executed and the scheduler on which to continue as arguments. When SG1 requested that a similar but different algorithm is to be used to implement scheduler affinity, `continues_on` was just replaced by `affine_on` with the same shape but the potential to get customized differently.

The scheduler used for affinity is the scheduler communicated via the `get_scheduler` query on the receiver's environment: the scheduler argument passed to the `affine_on` algorithm would need to match the scheduler obtained from `get_scheduler` query. In the context of the task coroutine this scheduler can be obtained via the promise type but in general it is actually not straight forward to get hold of this scheduler because the receiver and hence its associated scheduler is only provided by `connect`. It is much more reasonable to have `affine_on` only take the work, i.e., a sender, as argument and determine the scheduler to resume on from the receiver's environment in `connect`.

Thus, instead of using

```
affine_on(sndr, sch)
```

the algorithm is used just with the sender:

```
affine_on(sndr)
```

Note that this change implies that an operation state resulting from connecting `affine_on` to a receiver `rcvr` is started on the execution agent associated with the scheduler obtained from `get_scheduler(get_env(rcvr))`. The same requirement is also assumed to be met when starting the operation state resulting from connecting a task. While it is possible to statically detect whether the query is valid and provides a scheduler it cannot be detected if the scheduler matches the execution agent on which `start` was called. [P3718r0](#) proposed to add this exact requirement to `[exec.get.scheduler]` which is now moved to this proposal.

This change addresses [US 234-364 \(LWG4331\)](#).

§ 3.2 Scheduler An Operation Was Started On

The `on` and `affine_on` algorithms both use the scheduler returned from the `get_scheduler` query on the receiver to determine where to resume execution after some other work completed. When using these algorithms it is not always possible to determine which scheduler an operation gets started on when creating a work graph. Thus, the algorithms determine the scheduler from context. To actually do that, it needs to be required that the scheduler is made accessible when using these algorithms. [P3718r0](#) proposed that `get_scheduler` should be required to yield the scheduler an operation was started on.

It was pointed out that the meaning for the `get_scheduler` query isn't clear and it is used for two separate purposes:

1. The `get_scheduler` query is used to get a scheduler to schedule new work on.
2. The `get_scheduler` query is used to get the scheduler an operation was started on.

This dual use doesn't necessary align. It would be reasonable to add a new query, e.g., `get_start_scheduler` which is used to determine the scheduler an operation was started on. The requirement for getting the scheduler an operation was started on would be on the `get_start_scheduler` query rather than on the `get_scheduler` query. At the same time the `get_start_scheduler` query could be defaulted to use the `get_scheduler` query.

§ 3.3 Infallible Schedulers

The objective of `affine_on(sndr)` is to execute *sndr* and to complete on the execution agent on which the operation was started. Let *sch* be the scheduler obtained from `get_scheduler(get_env(rcvr))` where *rcvr* is the receiver used when connecting `affine_on(sndr)` (the discussion in this section also applies if the scheduler would be taken as a parameter, i.e., if the [previous change](#) isn't applied this discussion still applies). If connecting the result of `schedule(sch)` fails (i.e., `connect(schedule(sch), rcvr)` throws where *rcvr* is a suitable receiver), `affine_on` can avoid starting the main work and fail on the execution agent where it was started. Otherwise, if it obtained an operation state *os* from `connect(schedule(sch), rcvr)`, `affine_on` would start its main work and would start(*os*) on the execution agent where the main work completed. If `start(os)` is always successful, `affine_on` can achieve its objective. However, if this scheduling operation fails, i.e., it completes with `set_error(e)`, or if it gets cancelled, i.e., it completes with `set_stopped()`, the execution agent on which the scheduling operation resumes is unclear and `affine_on` cannot guarantee its promise. Thus, it seems reasonable to require that a scheduler used with `affine_on` is infallible, at least when used appropriately (i.e., when providing a receiver whose associated stop token is an `unstopable_token`).

The current working draft specifies 4 schedulers:

1. `inline_scheduler` which just completes with `set_value()` when `start()`ed, i.e., this scheduler is already infallible.
2. `task_scheduler` is a type-erased scheduler delegating to another scheduler. If the underlying scheduler is infallible, the only error case for `task_scheduler` is potential memory allocation during connect of its *ts-sender*. If `affine_on` creates an operation state for the scheduling operation during connect, it can guarantee that any necessary scheduling operation succeeds. Thus, this scheduler can be made infallible.
3. The `run_loop::run-loop-scheduler` is used by `run_loop`. The current specification allows the scheduling operation to fail with `set_error_t(std::exception_ptr)`. This permission allows an implementation to use `std::mutex` and `std::condition_variable` whose operations may throw. It is possible to implement the logic using atomic operations which can't throw. The `set_stopped()` completion is only used when the receiver's stop token, i.e. the result of `get_stop_token(get_env(rcvr))`, was stopped. This receiver is controlled by `affine_on`, i.e., it can provide a `never_stoptoken` and this scheduler won't complete with `set_stopped()`. If the `get_completion_signatures` for the corresponding sender takes the environment into account, this scheduler can also be made infallible.
4. The `parallel_scheduler` provides an interface to a replaceable implementation of a thread pool. The current interface allows `parallel_scheduler` to complete with `set_error_t(std::exception_ptr)` as well as with `set_stopped_t()`. It seems unlikely that this interface can be constrained to make it infallible.

In general it seems unlikely that all schedulers can be constrained to be infallible. As a result `affine_on` and, by extension, `task` won't be usable with all schedulers if `affine_on` insists on using only infallible schedulers. If there are fallible schedulers, there aren't any good options for using them with a `task`. Note that `affine_on` can fail and get cancelled (due to the main work failing or getting cancelled) but `affine_on` can still guarantee that execution resumes on the expected execution agent when it uses an infallible scheduler.

This change addresses [US 235-363 \(LWG4332\)](#). This change goes beyond the actual issue and clarifies that the scheduling operation used by `affine_on` needs to be always successful.

§ 3.3.1 Require Infallible Schedulers For `affine_on`

If `affine_on` promises in all cases that it resumes on the original scheduler it can only work with infallible schedulers. If a user wants to use a fallible scheduler with `affine_on` or `task` the scheduler will need to be adapted. The adapted scheduler can define what it means when the underlying scheduler fails. There are conceptually only two options (the exact details may vary) on how to deal with a failed scheduling operation:

1. The user can transform the scheduling failure into a call to `std::terminate`.

2. The user can consider resuming on an execution agent where the adapting scheduler can schedule to infallibly (e.g., the execution agent on which operation completed) but which is different from execution agent associated with the adapted scheduler to be suitable to continue running. In that case the scheduling operation would just succeed without necessarily running on the correct execution agent. However, there is no indication that scheduling to the adapted scheduler failed and the scheduler affinity may be impacted in this failure case.

The standard library doesn't provide a way to adapt schedulers easily. However, it can certainly be done.

§ 3.3.2 Allow Fallible Schedulers For `affine_on`

If the scheduler used with `affine_on` is allowed to fail, `affine_on` can't guarantee that it completes on the correct scheduler in case of an error completion. It could be specified that `affine_on` completes with `set_error(rcvr, scheduling_error{e})` when the scheduling operation completes with `set_error(r, e)` to make it detectable that it didn't complete on the correct scheduler. This situation is certainly not ideal but, at least, only affects the error completion and it can be made detectable.

A use of `affine_on` which always needs to complete on a specific scheduler is still possible: in that case the user will need to make sure that the used scheduler is infallible. The main issue here is that there is no automatic static checking whether that is the case.

§ 3.3.3 Considerations On Infallible Schedulers

In an ideal world, all schedulers would be infallible. It is unclear if that is achievable. If schedulers need to be allowed to be fallible, it may be viable to require that all standard library schedulers are infallible. As outlined above that should be doable for all current schedulers except, possibly, `parallel_scheduler`. So, the proposed change is to require schedulers to be infallible when being used with `affine_on` (and, thus, being used by `task`) and to change as many of the standard C++ libraries to be infallible as possible.

If constraining `affine_on` to only infallible schedulers turns out to be too strong, the constraint can be relaxed in a future revision of the standard by explicitly opting out of that constraints, e.g., using an additional argument. For `task` to make use of it, it too would need an explicit mechanisms to indicate that its `affine_on` use should opt out of the constraint, e.g., by adding a suitable `static` member to the environment template argument.

§ 3.4 `affine_on` Customization

Senders which don't cause the execution agent to be changed like `just` or the various queries should be able to customize `affine_on` to avoid unnecessary scheduling. Sadly, a proposal (P3206) to standardize properties which could be used to determine how a sender completes didn't make much progress, yet. An implementation can make use of similar techniques using an implementation-specific protocol. If a future standard defines a stan-

dard approach to determine the necessary properties the implementation can pick up on those.

The idea is to have `affine_on` define a `transform_sender(s)` member function which determines what sender should be returned. By default the argument is returned but if the child sender indicates that it doesn't actually change the execution agent the function would return the child sender. There are a number of senders for which this can be done:

- `just`, `just_error`, and `just_stopped`
- `read_env` and `write_env`
- `then`, `upon_error`, and `upon_stopped` if the child sender doesn't change the execution agent

The proposal is to define a `transform_sender` member which uses an implementation-specific property to determine that a sender completes on the same execution agent as the one it was started on. In addition, it is recommended that this property gets defined by the various standard library senders where it can make a difference.

This change addresses [US 232-366 \(LWG4329\)](#), although not in a way allowing application code to plug into this mechanism. Such an approach can be designed in a future revision of the standard.

§ 3.5 Removing `change_coroutine_scheduler`

The current working paper specifies `change_coroutine_scheduler` to change the scheduler used by the coroutine for scheduler affinity. It turns out that this use is somewhat problematic in two ways:

1. Changing the scheduler affects the coroutine until the end of the coroutine or until `change_coroutine_scheduler` is `co_awaited` again. It doesn't automatically reset. Thus, local variables constructed before `change_coroutine_scheduler(s)` was `co_awaited` were constructed on the original scheduler and are destroyed on the replaced scheduler.
2. The task's execution may finish on a different than the original scheduler. To allow symmetric transfer between two tasks each task needs to complete on the correct scheduler. Thus, the task needs to be prepared to change to the original scheduler before actually completing. To do so, it is necessary to know the original scheduler and also to have storage for the state needed to change to a different scheduler. It can't be statically detected whether `change_coroutine_scheduler(s)` is `co_awaited` in the body of a coroutine and, thus, the necessary storage and checks are needed even for tasks which don't use `change_coroutine_scheduler`.

If there were no way to change the scheduler it would still be possible to execute using a different scheduler, although not as direct: instead of using `co_await change_coroutine_scheduler(s)` to change the scheduler used for affinity to `s` a nested task executing on `s` could be `co_awaited`:


```
co_await ex::starts_on(s, [](parameters)->task<T, E> { logic }(arguments));
```

Using this approach the use of the scheduler `s` is clearly limited to the nested coroutine. The scheduler affinity is fully taken care of by the use of `affine_on` when `co_awaiting` work. There is no need to provide storage or checks needed for the potential of having a task return to the original scheduler if the scheduler isn't actually changed by a task.

The proposal is remove `change_coroutine_scheduler` and the possibility of changing the scheduler within a task. The alternative to controlling the scheduler used for affinity from within a task is a bit verbose. This need under the control of the coroutine is likely relatively rare. Replacing the used scheduler for an existing task by nesting it within `on(s, t)` or `starts_on(s, t)` is fairly straightforward.

This functionality was originally included because it is present for, at least, one of the existing libraries, although in a form which was recommended against. The existing use changes the scheduler of a coroutine when `co_awaiting` the result of `schedule(s)`; this exact approach was found to be fragile and surprising and the recommendation was to provide the functionality more explicit.

This change is not associated with any national body comment. However, it is still important to do! It isn't adding any new functionality but removes a problematic way to achieve something which can be better achieved differently. If this change is not made the inherent cost of having the possibility of having `change_routine_scheduler` can't be removed later without breaking existing code.

§ 3.6 `affine_on` Default Implementation

Using the previous discussion leads to a definition of `affine_on` which is quite different from effectively just using `continues_on`:

1. The class `affine_on` should define a `transform_sender` member function which returns the child sender if this child sender indicates via an implementation specific way that it doesn't change the execution agent. It should be recommended that some of the standard library sender algorithms (see above) to indicate that they don't change the execution agent.
2. The `affine_on` algorithm should only allow to get connected to a receiver `r` whose scheduler `sched` obtained by `get_scheduler(get_env(r))` is infallible, i.e., `get_completion_signatures(schedule(sched), e)` with an environment `e` where `get_stop_token(e)` yields `never_stop_token` returns `completion_signatures<set_value_t()>`.
3. When `affine_on` gets connected, the scheduling operation state needs to be created by connecting the scheduler's sender to a suitable receiver to guarantee that the completion can be scheduled on the execution agent. The stop token `get_stop_token(get_env(r))` for the receiver `r` used for this connect shall be an `unstoppable_token`. The child sender also needs to be connected with a receiver which

will capture the respective result upon completion and start the scheduling operation.

4. When the result operation state gets started it starts the operation state from the child operation.
5. Upon completion of the child operation the kind of completion and the parameters, if any, are stored. If this operation throws, the storage is set up to be as if `set_error(current_exception)` were called. Once the parameters are stored, the scheduling operation is started.
6. Upon completion of the scheduling operation, the appropriate completion function with the respective arguments is invoked.

This behavior is similar to `continues_on` but is subtly different with respect to when the scheduling operation state needs to be created and that any stop token from the receiver doesn't get forwarded. In addition `affine_on` is more constrained with respect to the schedulers it supports and the shape of the algorithm is different: `affine_on` gets the scheduler to execute on from the receiver it gets connected to.

This change addresses [US 233-365 \(LWG4330\)](#) and [US 236-362 \(LWG\)](#); the proposed resolution in this issue is incomplete).

§ 3.7 Name Change

The name `affine_on` isn't great. It may be worth giving the algorithm a better name.

§ 3.8 Use `get_start_scheduler` Properly

During the LEWG discussion on 2026-03-24 it was brought up that `get_start_scheduler` shouldn't fallback to `get_scheduler`. To make proper use of `get_start_scheduler` without that fallback, algorithms current using `get_scheduler` also need to use `get_start_scheduler` and advertise it appropriately:

- let* pass the first operation's `completion_scheduler<set_value_t>` as the `get_start_scheduler` for the second operation
- `starts_on` should advertise `get_start_scheduler`
- `task` should propagate `get_start_scheduler` instead of `get_scheduler`. Since it names the type, the type is also renamed `start_scheduler_type`.
- `sync_wait` advertises its scheduler using both `get_scheduler` and `get_start_scheduler`.

§ 4 Wording Changes

[Editor's note: This wording is relative to [N5032](#) with the changes from [P3826r3](#) applied.
]

[Editor's note: Add an exposition only concept *infallible-scheduler* to [exec.sched]:]

...

- 7 A scheduler type's destructor shall not block pending completion of any receivers connected to the sender objects returned from `schedule`.
- 8 The exposition-only *infallible-scheduler* concept defines the requirements of a scheduler type whose `schedule` asynchronous operation can only complete with `set_value` unless `stop` can be requested:

```
template <class Sch, class Env>
concept infallible-scheduler =
    scheduler<Sch> &&
    (same_as<completion_signatures<set_value_t()>,
     completion_signatures_of_t<decltype(schedule(declval<Sch>
        ())), Env>
        > ||
     (!unstoppable_token<stop_token_of_t<Env>> && (
        same_as<completion_signatures<set_value_t(), set_stopped_t()>,
        completion_signatures_of_t<decltype(schedule(declval<Sch>
        ())), Env>
        > ||
        same_as<completion_signatures<set_stopped_t(), set_value_t()>,
        completion_signatures_of_t<decltype(schedule(declval<Sch>
        ())), Env>
        >))
    >))
    >);
```

[Editor's note: Add `get_start_scheduler` to the synopsis in [execution.syn] after `get_scheduler` as follows:]

```
...
namespace std::execution {
    // [exec.queries], queries
    struct get_domain_t { unspecified };
    struct get_scheduler_t { unspecified };
    struct get_start_scheduler_t { unspecified };
    struct get_delegation_scheduler_t { unspecified };
    struct get_forward_progress_guarantee_t { unspecified };
    template<class CP0>
        struct get_completion_scheduler_t { unspecified };
    template<class CP0 = void>
        struct get_completion_domain_t { unspecified };
    struct get_await_completion_adaptor_t { unspecified };

    inline constexpr get_domain_t get_domain{};
    inline constexpr get_scheduler_t get_scheduler{};
    inline constexpr get_start_scheduler_t get_start_scheduler{};
    inline constexpr get_delegation_scheduler_t get_delegation_scheduler{};
    enum class forward_progress_guarantee;
        inline constexpr get_forward_progress_guarantee_t
            get_forward_progress_guarantee{};
```

```

template<class CP0>
constexpr get_completion_scheduler_t<CP0> get_completion_scheduler{};
template<class CP0 = void>
constexpr get_completion_domain_t<CP0> get_completion_domain{};
        inline      constexpr      get_await_completion_adaptor_t
        get_await_completion_adaptor{};
    ...
}

```

[Editor's note: Add a new section after [exec.get.scheduler] as follows:]

§ 4.0.1 `execution::get_start_scheduler` [exec.get.start.scheduler]

- 1 `get_start_scheduler` asks a queryable object for the scheduler an operation will be or was started on.
- 2 The name `get_start_scheduler` denotes a query object. For a subexpression `env`, `get_start_scheduler(env)` is expression-equivalent to `MANDATE-NO_THROW(AS-CONST(env).query(get_start_scheduler))`.

Mandates: If the expression above is well-formed, its type satisfies `scheduler`.

- 3 `forwarding_query(execution::get_start_scheduler)` is a core constant expression and has value `true`.
- 4 Given subexpressions `sndr` and `rcvr` such that `sender_to<decltype((sndr)), decltype((rcvr))>` is true and the expression `get_start_scheduler(get_env(rcvr))` is well-formed, an operation state that is the result of calling `connect(sndr, rcvr)` shall, if it is started, be started on an execution agent associated with the scheduler `get_start_scheduler(get_env(rcvr))`.

[Editor's note: Add two paragraphs at the start of [exec.snd.expos] giving implementations permission to add members `affine_on` and `as_awaitable` to standard library sender types:]

- ? Given an expression `sndr`, whose type is any sender type defined in the standard library, it is unspecified whether the expression `sndr.affine_on()` is well-formed. If that expression is well-formed, then the evaluation thereof meets the semantic requirements of the `affine_on` [exec.affine.on] algorithm.
- ? Given an expression `sndr`, whose type is any sender type defined in the standard library, and an expression `p`, whose type is a promise type, it is unspecified whether the expression `sndr.as_awaitable(p)` is well-formed. If that expression is well-formed, then the evaluation thereof meets the semantic requirements of the `as_awaitable` [exec.as.awaitable] algorithm.

[Editor's note: Change [exec.snd.expos] paragraph 8 to have *SCHED-ENV* use `get_start_scheduler` instead of `get_scheduler`:]

- 8 `SCHED-ENV(sch)` is an expression `o2` whose type satisfies *queryable* such that `o2.query(get_schedulerget_start_scheduler)` is a prvalue with the same type and value as `sch`, and such that `o2.query(get_domain)` is expression-equivalent to `sch.query(get_domain)`.

[Editor's note: The specification of `on [exec.on]` shouldn't use `write_env` as it does, i.e., this change removes these (not removing them was an oversight in P3826).

In addition, if `get_start_scheduler` is introduced change how `on` gets its scheduler in `[exec.on]`, i.e., change the use from `get_scheduler` to use `get_start_scheduler`:]

...

- 8 Otherwise, the expression `on.transform_sender(set_value, out_sndr, env)` has effects equivalent to:

```
auto&& [_ , data, child] = out_sndr;
if constexpr (scheduler<decltype(data)>) {
    auto orig_sch =
        call-with-default(get_schedulerget_start_scheduler, not-a-scheduler(),
            env);

    return continues_on(
        starts_on(std::forward_like<OutSndr>(data), std::forward_like<OutSndr>(
            child)),
        std::move(orig_sch));
} else {
    auto& [sch, closure] = data;
    auto orig_sch = call-with-default(
        get_completion_scheduler<set_value_t>,    not-a-scheduler(),
        get_env(child), env);

    return continues_on(
        std::forward_like<OutSndr>(closure)(
            continues_on(
                std::forward_like<OutSndr>(child),
                sch)),
        orig_sch);
}
```

- 9 Let `out_sndr` be a subexpression denoting a sender returned from `on(sch, sndr)` or one equal to such, and let `OutSndr` be the type `decltype((out_sndr))`. Let `out_rcvr` be a subexpression denoting a receiver that has an environment of type `Env` such that `sender_in<OutSndr, Env>` is `true`. Let `op` be an lvalue referring to the operation state that results from connecting `out_sndr` with `out_rcvr`. Calling `start(op)` shall

- (9.1) — remember the current scheduler; which is obtained by ~~get_scheduler~~get_start_scheduler(get_env(rcvr));
- (9.2) — start `sndr` on an execution agent belonging to `sch`'s associated execution resource;
- (9.3) — upon `sndr`'s completion, transfer execution back to the execution resource associated with the scheduler remembered in step 1; and

- (9.4) — forward `sndr`'s async result to `out_rcvr`.

If any scheduling operation fails, an error completion on `out_rcvr` shall be executed on an unspecified execution agent.

[Editor's note: Change `[exec.sync.wait]` p2 to also provide a `get_start_scheduler` query:]

Let `sync-wait-env` be the following exposition-only class type:

```
namespace std::this_thread {
    struct sync-wait-env {
        execution::run_loop* loop;                // exposition only

        auto query(execution::get_scheduler_t) const noexcept {
            return loop->get_scheduler();
        }

        auto query(execution::get_start_scheduler_t) const noexcept {
            return loop->get_scheduler();
        }

        auto query(execution::get_delegation_scheduler_t) const noexcept {
            return loop->get_scheduler();
        }
    };
}
```

[Editor's note: Change `[exec.as.awaitable]` paragraph 7 such that it tries to locate a customization for `as_awaitable` on the transformed nested sender.]

- 7 `as_awaitable` is a customization point object. For subexpressions `expr` and `p` where `p` is an lvalue, `Expr` names the type `decltype((expr))` and `Promise` names the type `decay_t<decltype((p))>`, `as_awaitable(expr, p)` is expression-equivalent to, except that the evaluations of `expr` and `p` are indeterminately sequenced:

- (7.1) — `expr.as_awaitable(p)` if that expression is well-formed.

Mandates: `is-awaitable<A, Promise>` is `true`, where `A` is the type of the expression above.

- (7.?) — Otherwise, `adapt-for-await-completion(transform_sender(expr, get_env(p))).as_awaitable(p)` if this expression is well-formed, `sender_in<Expr, env of t<Promise>>` is true, and `single-sender-value-type<Expr, env of t<Promise>>` is well-formed, except that `p` is only evaluated once.

- (7.2) — Otherwise, `(void(p), expr)` if `decltype(GET-AWAITER(expr))` satisfies `isawaiter<Promise>`.

- (7.3) — ~~Otherwise, `sender_awaitable{adapted_expr, p}` if~~

~~has-queryable-await-completion-adaptor<Expr>~~

~~and~~

~~awaitable-sender<decltype((adapted-expr)), Promise>~~

~~are both satisfied, where adapted-expr is
get-await-completion-adaptor(get-env(expr))(expr), except that expr is eval-
uated only once.~~

(7.4) — Otherwise, ~~sender-awaitable{expr, p} if awaitable-sender<Expr, Promise> is
true:~~ [sender-awaitable{adapt-for-await-completion(transform_sender(expr, get_env(p))), p} if sender_in<Expr, env_of_t<Promise>> is
true and single-sender-value-type<Expr, env_of_t<Promise>> is well-
formed], except that p is only evaluated once.{.add}

(7.5) — Otherwise, (void(p), expr).

8 adapt-for-await-completion(s) is expression-equivalent to

- get-await-completion-adaptor(get_env(s))(s) if that is well-formed, except
that s is evaluated only once,
- otherwise, s.

[Editor's note: Change [exec.affine.on] to use only one parameter, require an infallible
scheduler from the receiver, and add a default implementation which allows customization
of affine_on for child senders. The algorithm affine_on should use get_start_sched-
uler to get the start scheduler:]

1 affine_on adapts a sender into one that completes on a ~~specified-scheduler~~ receiver's
scheduler. If the algorithm determines that the adapted sender already completes on the
correct scheduler it can avoid any scheduling operation.

2 The name affine_on denotes a pipeable sender adaptor object. For a subexpressions ~~s-sch
and~~ sndr, if ~~decltype((sch)) does not satisfy scheduler, or~~ decltype((sndr)) does not
satisfy sender, affine_on(sndr, ~~sch~~) is ill-formed.

3 Otherwise, the expression affine_on(sndr, ~~sch~~) is expression-equivalent to make-
sender(affine_on, ~~sch~~env<>(), sndr).

? For a subexpression sch whose type models scheduler, let UNSTOPPABLE-
SCHEDULER(sch) be an expression e whose type models scheduler such that:

(?.1) — schedule(e) is expression-equivalent to unstoppable(schedule(sch)).

(?.2) — For any query object q and pack of subexpressions args..., e.query(q, args...) is
expression-equivalent to sch.query(q, args...).

(?.3) — The expression e == UNSTOPPABLE-SCHEDULER(other) is expression-equivalent to
sch == other.

- ? Let `sndr` and `ev` be subexpressions such that `Sndr` is `decltype((sndr))`. If `sender-for<Sndr, affine_on_t>` is false, then the expression `affine_on.transform_sender(sndr, ev)` is ill-formed; otherwise, it is equal to:

```
auto&[_ , _ , child] = sndr;
if constexpr (requires{ std::forward_like<Sndr>(child).affine_on(); })
    return std::forward_like<Sndr>(child).affine_on();
else
    return continues_on(std::forward_like<Sndr>(child), UNSTOPPABLE-
        SCHEDULER(get_start_scheduler(ev)));
```

- ? *Recommended Practice:* Implementations should provide `affine_on` member functions for senders that are known to resume on the scheduler where they were started. Example senders for which that is the case are `just`, `just_error`, `just_stopped`, `read_env`, and `write_env`.

- 5 Let `out_sndr` be a subexpression denoting a sender returned from `affine_on(sndr, sch)` or one equal to such, and let `OutSndr` be the type `decltype((out_sndr))`. Let `out_rcvr` be a subexpression denoting a receiver that has an environment of type `Env` ~~such that `sender_in<OutSndr, Env>` is true~~. If `get_start_scheduler(get_env(out_rcvr))` is ill-formed or does not satisfy `infallible_scheduler<Env>` then evaluation of the expression `get_completion_signatures<OutSndr, Env>()` exits with an exception. Let `op` be an lvalue referring to the operation state that results from connecting `out_sndr` to `out_rcvr`. Calling `start(op)` will start `sndr` on the current execution agent and execute completion operations on `out_rcvr` on an execution agent of the execution resource associated with ~~`sch`~~`sch`. If the current execution resource is the same as the execution resource associated with ~~`sch`~~`sch`, the completion operation on `out_rcvr` may be called before `start(op)` completes. ~~If scheduling onto `sch` fails, an error completion on `out_rcvr` shall be executed on an unspecified execution agent.~~

[Editor's note: Remove `change_coroutine_scheduler` from `[execution.syn]`:]

```
namespace std::execution {
    ...
    // [exec.task.scheduler], task scheduler
    class task_scheduler;

    template<class E>
    struct with_error {
        using type = remove_cvref_t<E>;
        type error;
    };
    template<class E>
    with_error(E) -> with_error<E>;
```

```
template<scheduler Sch>
struct change_coroutine_scheduler {
    using type = remove_cvref_t<Sch>;
    type scheduler;
};
```



```

template<scheduler Sch>
    change_coroutine_scheduler(Sch) -> change_coroutine_scheduler<Sch>;

// [exec.task], class template task
template<class T, class Environment>
    class task;
...
}

```

In `[task.class]` change `scheduler_type` to `start_scheduler_type`:

```

namespace std::execution {
    template<class T = void, class Environment = env<>>
    class task {
        // [task.state]
        template<receiver Rcvr>
            class state;                                // exposition only

    public:
        using sender_concept = sender_t;
        using completion_signatures = see below;
        using allocator_type = see below;
        using scheduler_typestart_scheduler_type = see below;
        using stop_source_type = see below;
            using stop_token_type = decltype(declval<stop_source_type>
                ().get_token());
        using error_types = see below;

        // [task.promise]
        class promise_type;

        task(task&&) noexcept;
        ~task();

        template<receiver Rcvr>
            state<Rcvr> connect(Rcvr&& rcvr) &&;

    private:
        coroutine_handle<promise_type> handle;          // exposition only
    };
}

```

- 1 `task<T, E>` models sender ([exec.snd]) if `T` is `void`, a reference type, or a *cv*-unqualified non-array object type and `E` is a class type. Otherwise a program that instantiates the definition of `task<T, E>` is ill-formed.
- 2 The nested types of `task` template specializations are determined based on the `Environment` parameter:
 - (2.1) — `allocator_type` is `Environment::allocator_type` if that *qualified-id* is valid and denotes a type, `allocator<byte>` otherwise.
 - (2.2) — ~~`scheduler_type`~~`start_scheduler_type` is `Environment::scheduler_typestart_scheduler_type if that qualified-id is valid and denotes a type, task_scheduler otherwise.`

- (2.3) — `stop_source_type` is `Environment::stop_source_type` if that *qualified-id* is valid and denotes a type, `inplace_stop_source` otherwise.
- (2.4) — `error_types` is `Environment::error_types` if that *qualified-id* is valid and denotes a type, `completion_signatures<set_error_t(exception_ptr)>` otherwise.
- 3 A program is ill-formed if `error_types` is not a specialization of `execution::completion_signatures` or if the template arguments of that specialization contain an element which is not of the form `set_error_t(E)` for some type `E`.
- 4 The type alias `completion_signatures` is a specialization of `execution::completion_signatures` with the template arguments (in unspecified order):
 - (4.1) — `set_value_t()` if `T` is `void`, and `set_value_t(T)` otherwise;
 - (4.2) — template arguments of the specialization of `execution::completion_signatures` denoted by `error_types`; and
 - (4.3) — `set_stopped_t()`.
- 5 `allocator_type` shall meet the *Cpp17Allocator* requirements.

[Editor's note: Change the scheduler propagation to start to use `get_start_scheduler` in `[task.state]` p4:]

```
void start() & noexcept;
```

- 4 *Effects:* Let `prom` be the object `handle.promise()`. Associates `STATE(prom)`, `RCVR(prom)`, and `SCHED(prom)` with `*this` as follows:
 - (4.1) — `STATE(prom)` is `*this`.
 - (4.2) — `RCVR(prom)` is `rcvr`.
 - (4.3) — `SCHED(prom)` is the object initialized with `scheduler_typestart_scheduler_type(get_schedulerget_start_scheduler(get_env(rcvr)))` if that expression is valid and `scheduler_typestart_scheduler_type()` otherwise. If neither of these expressions is valid, the program is ill-formed.

Let `st` be `get_stop_token(get_env(rcvr))`. Initializes `prom.token` and `prom.source` such that

- (4.4) — `prom.token.stop_requested()` returns `st.stop_requested()`;
- (4.5) — `prom.token.stop_possible()` returns `st.stop_possible()`; and
- (4.6) — for types `Fn` and `Init` such that both `invocable<Fn>` and `constructible_from<Fn, Init>` are modeled, `stop_token_type::callback_type<Fn>` models *stoppable-callback-for*<`Fn`, `stop_token_type`, `Init`>.

After that invokes `handle.resume()`.

[Editor's note: Use `get_start_scheduler` instead of `get_scheduler` when providing an environment to the `co_awaited` sender in [task.state] p16:]

```
unspecified get_env() const noexcept;
```

16 *Returns:* An object `env` such that queries are forwarded as follows:

- (16.1) — `env.query(get_schedulerget_start_scheduler)` returns ~~`scheduler_type`~~`start_scheduler_type(SCHED(*this))`.
- (16.2) — `env.query(get_allocator)` returns `alloc`.
- (16.3) — `env.query(get_stop_token)` returns `token`.
- (16.4) — For any other query `q` and arguments `a...` a call to `env.query(q, a...)` returns `STATE(*this).environment.query(q, a...)` if this expression is well-formed and `forwarding_query(q)` is well-formed and is `true`. Otherwise `env.query(q, a...)` is ill-formed.

[Editor's note: Adjust the use of `affine_on` and remove `change_coroutine_scheduler` from [task.promise]:]

```
namespace std::execution {  
    template<class T, class Environment>  
    class task<T, Environment>::promise_type {  
    public:  
        ...  
  
        template<class A>  
        auto await_transform(A&& a);  
    }  
};
```

```
template<class Sch>  
    auto await_transform(change_coroutine_scheduler<Sch> sch);
```

```
unspecified get_env() const noexcept;  
  
    ...  
};  
};
```

...

```
template<sender Sender>  
    auto await_transform(Sender&& sndr) noexcept;
```

- 9 *Returns:* If `same_as<inline_scheduler, scheduler_typestart_scheduler_type>` is `true` returns `as_awaitable(std::forward<Sender>(sndr), *this)`; otherwise returns `as_awaitable(affine_on(std::forward<Sender>(sndr), SCHED(*this)), *this)`.

```
template<class Sch>  
    auto await_transform(change_coroutine_scheduler<Sch> sch) noexcept;
```

- 10 *Effects:* Equivalent to:

```
return          await_transform(just(exchange(SCHED(*this),
        scheduler_type(sch.scheduler))), *this);
```

```
void unhandled_exception();
```

- 11 *Effects:* If the signature `set_error_t(exception_ptr)` is not an element of `error_types`, calls `terminate()` ([except.terminate]). Otherwise, stores `current_exception()` into `errors`.

...

[Editor's note: In [exec.task.scheduler] change the constructor of `task_scheduler` to require that the scheduler passed is infallible]

```
template<class Sch, class Allocator = allocator<void>>
    requires(!same_as<task_scheduler,      remove_cvref_t<Sch>>)    &&
    scheduler<Sch>
explicit task_scheduler(Sch&& sch, Allocator alloc = {});
```

- ? *Mandates:* `Sch` satisfies *infallible-scheduler*<env>>.
- 2 *Effects:* Initialize `sch_` with `allocate_shared<remove_cvref_t<Sch>>(alloc, std::forward<Sch>(sch))`.
- 3 *Recommended practice:* Implementations should avoid the use of dynamically allocated memory for small scheduler objects.
- 4 *Remarks:* Any allocations performed by construction of *ts-sender* or *state* objects resulting from calls on **this* are performed using a copy of `alloc`.

[Editor's note: In [exec.task.scheduler] change the *ts-sender* completion signatures to indicate that `task_scheduler` is infallible:]

```
8 namespace std::execution {
    class task_scheduler::ts-sender {      // exposition only
    public:
        using sender_concept = sender_t;

        template<receiver Rcvr>
            state<Rcvr> connect(Rcvr&& rcvr) &&;
    };
}
```

ts-sender is an exposition-only class that models sender ([exec.snd]) and for which `completion_signatures_of_t<ts-sender, E>` denotes completion signatures<set value t()> if unstoppable token<stop token of t<E>> is true, and otherwise completion signatures<set value t(), set stopped t()>.

```
completion_signatures<
    set_value_t(),
    set_error_t(error_code),
```

```
set_error_t(exception_ptr),
set_stopped_t()>
```

[Editor's note: In [exec.run.loop.types] change the paragraph defining the completion signatures:]

...

```
class run-loop-sender;
```

- 6 *run-loop-sender* is an exposition-only type that satisfies sender. Let E be the type of an environment. If unstoppable token<stop token of t<E>> is true, then completion_signatures_of_t<run-loop-sender, _E> is

```
completion_signatures<set_value_t(),    set_error_t(exception_ptr),
set_stopped_t()>
```

```
completion_signatures<set_value_t()>
```

Otherwise it is

```
completion_signatures<set_value_t(), set_stopped_t()>
```

- 7 An instance of *run-loop-sender* remains valid until the end of the lifetime of its associated *run_loop* instance.

...

[Editor's note: Change [exec.counting.scopes.general] p4 to use `get_start_scheduler` instead of `get_scheduler`:]

- 4 The exposition-only class template *impls-for* ([exec.snd.expos]) is specialized for *scope-join-t* as follows:

```
namespace std::execution {
    template<>
    struct impls-for<scope-join-t> : default-impls {
        template<class Scope, class Rcvr>
        struct state {                                // exposition only
            struct rcvr-t {                            // exposition only
                using receiver_concept = receiver_t;

                Rcvr& rcvr;                            // exposition only

                void set_value() && noexcept {
                    execution::set_value(std::move(rcvr));
                }

                template<class E>
                void set_error(E&& e) && noexcept {
                    execution::set_error(std::move(rcvr), std::forward<E>(e));
                }

                void set_stopped() && noexcept {
```

```

        execution::set_stopped(std::move(rcvr));
    }

    decltype(auto) get_env() const noexcept {
        return execution::get_env(rcvr);
    }
};

using sched_sender =                // exposition only
    decltype(schedule(get_schedulerget_start_sched-
uler(get_env(declval<Rcvr&>()))));
using op_t =                        // exposition only
    connect_result_t<sched_sender, rcvr_t>;

Scope* scope;                      // exposition only
Rcvr& receiver;                    // exposition only
op_t op;                           // exposition only

state(Scope* scope, Rcvr& rcvr)     // exposition only
    noexcept(nothrow_callable<connect_t, sched_sender, rcvr_t>)
    : scope(scope),
      receiver(rcvr),
      op(connect(schedule(get_schedulerget_start_sched-
uler(get_env(rcvr))), rcvr_t(rcvr))) {}

void complete() noexcept {          // exposition only
    start(op);
}

void complete-inline() noexcept {    // exposition only
    set_value(std::move(receiver));
}
};

static constexpr auto get_state =    // exposition only
    [<class Rcvr>(auto&& sender, Rcvr& receiver)
     noexcept(is_nothrow_constructible_v<state<Rcvr>, data_type<de-
     cltype(sender)>, Rcvr&>) {
        auto[_ , self] = sender;
        return state(self, receiver);
    }];

static constexpr auto start =        // exposition only
    [<auto& s, auto& >() noexcept {
        if (s.scope->start_join_sender(s))
            s.complete-inline();
    }];
};
}

```